main lab-k-means-clustering / lab-k-means-clustering.ipynb

sandrabosk initial commit d722dbd · last year

887 lines (887 loc) · 23.5 KB

Preview Code Blame Raw

K-Means Clustering

Estimated time needed: **25-30** minutes

Objectives

After completing this lab you will be able to:

- Use scikit-learn's K-Means Clustering to cluster data

Introduction

There are many models for **clustering** out there. In this notebook, we will be presenting the model that is considered one of the simplest models amongst them. Despite its simplicity, the **K-means** is vastly used for clustering in many data science applications, it is especially useful if you need to quickly discover insights from **unlabeled data**. In this notebook, you will learn how to use k-Means for customer segmentation.

Some real-world applications of k-means:

- Customer segmentation
- Understand what the visitors of a website are trying to accomplish
- Pattern recognition
- Machine learning
- Data compression

In this notebook we practice k-means clustering with 2 examples:

- k-means on a random generated dataset
- Using k-means for customer segmentation

Table of contents

- k-Means on a randomly generated dataset
    1. Setting up K-Means
    2. Creating the Visual Plot
- Customer Segmentation with K-Means
    1. Pre-processing
    2. Modeling
    3. Insights

Import the Libraries

Let's first import the required libraries. Also run **%matplotlib inline** since we will be plotting in this section.

In [ ]:
```
#you are running the lab in your  browser, so we will install the libra
import piplite
await piplite.install(['pandas'])
await piplite.install(['matplotlib'])
await piplite.install(['scipy'])
await piplite.install(['seaborn'])
```

In [ ]:
```
import random
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
%matplotlib inline
```

Download the Data

In [ ]:
```
from pyodide.http import pyfetch

async def download(url, filename):
    response = await pyfetch(url)
    if response.status == 200:
        with open(filename, "wb") as f:
            f.write(await response.bytes())
```

k-Means on a randomly generated dataset

Let's create our own dataset for this lab!

First we need to set a random seed. Use **numpy's random.seed()** function, where the seed will be set to **0**.

In [ ]:
```
np.random.seed(0)
```

Next we will be making *random clusters* of points by using the **make_blobs** class. The **make_blobs** class can take in many inputs, but we will be using these specific ones.

Input

- **n_samples**: The total number of points equally divided among clusters.
  - Value will be: 5000
- **centers**: The number of centers to generate, or the fixed center locations

- Value will be: [[4, 4], [-2, -1], [2, -3],[1,1]]
- **cluster_std**: The standard deviation of the clusters.
  - Value will be: 0.9

Output

- **X**: Array of shape [n_samples, n_features]. (Feature Matrix)
  - The generated samples.
- **y**: Array of shape [n_samples]. (Response Vector)
  - The integer labels for cluster membership of each sample.

In [ ]:
```
X, y = make_blobs(n_samples=5000, centers=[[4,4], [-2, -1], [2, -3], [1
```

Display the scatter plot of the randomly generated data.

In [ ]:
```
plt.scatter(X[:, 0], X[:, 1], marker='.')
```

Setting up K-Means

Now that we have our random data, let's set up our K-Means Clustering.

The KMeans class has many parameters that can be used, but we will be using these three:

- **init**: Initialization method of the centroids.
  - Value will be: "k-means++"
  - k-means++: Selects initial cluster centers for k-mean clustering in a smart way to speed up convergence.
- **n_clusters**: The number of clusters to form as well as the number of centroids to generate.
  - Value will be: 4 (since we have 4 centers)
- **n_init**: Number of time the k-means algorithm will be run with different centroid seeds. The final results will be the best output of n_init consecutive runs in terms of inertia.
  - Value will be: 12

Initialize KMeans with these parameters, where the output parameter is called **k_means**.

In [ ]:
```
k_means = KMeans(init = "k-means++", n_clusters = 4, n_init = 12)
```

Now let's fit the KMeans model with the feature matrix we created above, **X** .

In [ ]:
```
k_means.fit(X)
```

Now let's grab the labels for each point in the model using KMeans' **.labels_** attribute and save it as **k_means_labels**

attribute and save it as **k_means_labels** :

In [ ]:
```
k_means_labels = k_means.labels_
k_means_labels
```

We will also get the coordinates of the cluster centers using KMeans' **.cluster_centers_** and save it as **k_means_cluster_centers** .

In [ ]:
```
k_means_cluster_centers = k_means.cluster_centers_
k_means_cluster_centers
```

Creating the Visual Plot

So now that we have the random data generated and the KMeans model initialized, let's plot them and see what it looks like!

Please read through the code and comments to understand how to plot the model.

In [ ]:
```
# Initialize the plot with the specified dimensions.
fig = plt.figure(figsize=(6, 4))

# Colors uses a color map, which will produce an array of colors based
# the number of labels there are. We use set(k_means_labels) to get the
# unique labels.
colors = plt.cm.Spectral(np.linspace(0, 1, len(set(k_means_labels))))

# Create a plot
ax = fig.add_subplot(1, 1, 1)

# For loop that plots the data points and centroids.
# k will range from 0-3, which will match the possible clusters that ea
# data point is in.
for k, col in zip(range(len([[4,4], [-2, -1], [2, -3], [1, 1]])), colo

    # Create a list of all data points, where the data points that are
    # in the cluster (ex. cluster 0) are labeled as true, else they are
    # labeled as false.
    my_members = (k_means_labels == k)

    # Define the centroid, or cluster center.
    cluster_center = k_means_cluster_centers[k]

    # Plots the datapoints with color col.
    ax.plot(X[my_members, 0], X[my_members, 1], 'w', markerfacecolor=c

    # Plots the centroids with specified color, but with a darker outli
    ax.plot(cluster_center[0], cluster_center[1], 'o', markerfacecolor=

# Title of the plot
ax.set_title('KMeans')

# Remove x-axis ticks
ax.set_xticks(())

# Remove y-axis ticks
ax.set_yticks(())

# Show the plot
```

```
plt.show()
```

Practice

Try to cluster the above dataset into 3 clusters.
Notice: do not generate the data again, use the same dataset as above.

In [ ]:
```
# write your code here
```

Click here for the solution

```
k_means3 = KMeans(init = "k-means++", n_clusters = 3, n_init =
12)
k_means3.fit(X)
fig = plt.figure(figsize=(6, 4))
colors = plt.cm.Spectral(np.linspace(0, 1,
len(set(k_means3.labels_))))
ax = fig.add_subplot(1, 1, 1)
for k, col in zip(range(len(k_means3.cluster_centers_)),
colors):
    my_members = (k_means3.labels_ == k)
    cluster_center = k_means3.cluster_centers_[k]
    ax.plot(X[my_members, 0], X[my_members, 1], 'w',
markerfacecolor=col, marker='.')
    ax.plot(cluster_center[0], cluster_center[1], 'o',
markerfacecolor=col,  markeredgecolor='k', markersize=6)
plt.show()
```

Customer Segmentation with K-Means

Imagine that you have a customer dataset, and you need to apply customer segmentation on this historical data. Customer segmentation is the practice of partitioning a customer base into groups of individuals that have similar characteristics. It is a significant strategy as a business can target these specific groups of customers and effectively allocate marketing resources. For example, one group might contain customers who are high-profit and low-risk, that is, more likely to purchase products, or subscribe for a service. A business task is to retain those customers. Another group might include customers from non-profit organizations and so on.

Let's download the datasetfrom IBM Object Storage. **Did you know?** When it comes to Machine Learning, you will likely be working with large datasets. As a business, where can you host your data? IBM is offering a unique opportunity for businesses, with 10 Tb of IBM Cloud Object Storage: Sign up now for free

In [ ]:
```
path='https://cf-courses-data.s3.us.cloud-object-storage.appdomain.clou
```

Load Data From CSV File

Before you can work with the data, you must use the URL to get the Cust_Segmentation.csv.

In [ ]:
```
await download(path, "Cust_Segmentation.csv")
filename ="Cust_Segmentation.csv"
```

we create a pandas dataframe

In [ ]:
```
import pandas as pd
cust_df = pd.read_csv("Cust_Segmentation.csv")
cust_df.head()
```

Pre-processing

As you can see, **Address** in this dataset is a categorical variable. The k-means algorithm isn't directly applicable to categorical variables because the Euclidean distance function isn't really meaningful for discrete variables. So, let's drop this feature and run clustering.

In [ ]:
```
df = cust_df.drop('Address', axis=1)
df.head()
```

Normalizing over the standard deviation

Now let's normalize the dataset. But why do we need normalization in the first place? Normalization is a statistical method that helps mathematical-based algorithms to interpret features with different magnitudes and distributions equally. We use **StandardScaler()** to normalize our dataset.

In [ ]:
```
from sklearn.preprocessing import StandardScaler
X = df.values[:,1:]
X = np.nan_to_num(X)
Clus_dataSet = StandardScaler().fit_transform(X)
Clus_dataSet
```

Modeling

In our example (if we didn't have access to the k-means algorithm), it would be the same as guessing that each customer group would have certain age, income, education, etc, with multiple tests and experiments. However, using the K-means clustering we can do all this process much easier.

Let's apply k-means on our dataset, and take a look at cluster labels.

In [ ]:
```
clusterNum = 3
k_means = KMeans(init = "k-means++", n_clusters = clusterNum, n_init
k_means.fit(X)
labels = k_means.labels_
print(labels)
```

Insights

We assign the labels to each row in the dataframe.

In [ ]:
```
df["Clus_km"] = labels
df.head(5)
```

We can easily check the centroid values by averaging the features in each cluster.

In [ ]:
```
df.groupby('Clus_km').mean()
```

Now, let's look at the distribution of customers based on their age and income:

In [ ]:
```
area = np.pi * ( X[:, 1])**2
plt.scatter(X[:, 0], X[:, 3], s=area, c=labels.astype(np.float), alph
plt.xlabel('Age', fontsize=18)
```